

```
// Example Using TryParse:
if (double.TryParse(t1, out double d1) && double.TryParse(t2, out double d2))
{
    double result = d2 - d1;
    resultTextBox.Text = $"{result}";
}
```

```
// Looping through form control:
foreach (var control in this.Controls)
{
    if (control is TextBox textBox)
    {
        textBox.Clear();
    }
}
```

```
try // Try Catch and showing MessageBox
{
    double d1 = double.Parse(t1);
    double d2 = double.Parse(t2);
    firstTextBox.Text = $"{d2}";
    secondTextBox.Text = $"{d1}";
}
catch (Exception ex) { MessageBox.Show(ex.Message); }
```

```
List<Car> cars = new List<Car>(); // creating a list
Car car1 = new Car("Toyota", "Camry", 50, 15); // instantiate car
cars.Add(car1); // add to list
Car retrievedCar = cars.Find(c => c.Make == "Toyota"); // find specific item using Lambda;
cars.Remove(car1); // remove from list
```

```
employees.Insert(0, new Employee("Jess", "Pan", 12333)); // insert into list at specific index
```

```
Employee emp = new Employee("Bob", "Joe", 123123);
employees.Add(emp);
bool contains = employees.Contains(emp); // check to see if something exists in list
```

```
employees.RemoveAt(1); // remove item at specific index
```

```
var results = // use var, assume list of employees exists
from e in employees
where e.MonthlySalary >= 6000 // use can set conditions. Also access object properties
select e; // grab the item. then use a for loop through results
```

```
var names =
from e in employees
where e.FirstName == "Sarah" && e.LastName == "Brown"
select new { e.FirstName, e.LastName, e.MonthlySalary }; // select individual stuff
```

```
listBox1.Items.Clear(); // clear listbox
listBox1.Items.Add(t); // add to listbox
```

```
groupBox1.Enabled = true; // enable groupbox
groupBox1.Enabled = false; // disable groupbox
```

```
checkingRadioButton.Checked = true; // radiobutton checked
checkingRadioButton.Checked = false; // radiobutton not checked
```

```
if (listBox1.SelectedIndex == -1) // check if no item is selected
{
    label1.ForeColor = Color.Red;
    label1.Text = "Status: No Item Selected";
    return;
}

int index = listBox1.SelectedIndex; // get item at selected index for listbox
string item = listBox1.Items[index].ToString(); // get content of listbox item at specific index (string)
listBox1.Items.RemoveAt(index); // remove item at selected index (int) for listbox
if ((index + 1) <= listBox1.Items.Count)
{
    index++;
}

listBox1.Items.Insert(index, item); // insert into listbox at index pos, then supply content (string)
listBox1.SelectedIndex = index; // reselect index
```

```
// this example uses LINQ and orderby emp.LastName for the employees list (LIST OF OBJECTS).
var results = // results is a collection that we can look over based off of what we get
from emp in employees
orderby emp.LastName
select emp;
```

```
// looping over result set
foreach (var result in results)
{
    // we can use lambda as well. This will sort the employees list in place I think
    employees.Sort((e1, e2) => e1.Salary.CompareTo(e2.Salary));
}
```

```
foreach (Employee emp in employees)
{
    employeeListBox.Items.Add(emp);
}
```

```
public partial class Form1 : Form
{
    SqlConnection Connection; // globally declare this, but don't instantiate yet.
    string connectionString = @"Data Source=.\\SQLEXPRESS;Initial Catalog=toms_plumbing;Integrated Security=True;";
```

```
// INSIDE public Form1() AND wrap inside Try Catch
Connection = new SqlConnection(); // instantiate connection here
Connection.ConnectionString = connectionString; // do this
Connection.Open(); // open connection
```

```
SqlCommand command = new SqlCommand("SELECT * FROM employees", Connection); // simple query everything
using (SqlDataReader reader = command.ExecuteReader())
{
    while (reader.Read()) // access columns with [] and create objects, add to listbox
    {
        int ID = int.Parse(reader["employee_id"].ToString());
        string firstName = reader["first_name"].ToString();
        string lastName = reader["last_name"].ToString();
        int departmentID = int.Parse(reader["department_id"].ToString());
        string gender = reader["gender"].ToString();
        double salary = double.Parse(reader["salary"].ToString());
        string prov = reader["province"].ToString();
        Employee emp = new Employee(ID, firstName, lastName, departmentID, gender, salary, prov);
        employees.Add(emp);
        employeeListBox.Items.Add(emp);
    }
}
```

```
public string Make { get; private set; } // public get, private set
2 references
public string Model { private get; set; } // private get, public set
3 references
private double Fuel { get; set; } // private get, private set
4 references
public double Mileage { get; set; } // public get, public set
```

```
private decimal monthlySalary; // using backing variables
2 references
public decimal MonthlySalary
{
    get { return monthlySalary; }
    private set
    {
        if (value >= 0)
            monthlySalary = value;
    }
}

1 reference
public Employee(string firstName, string lastName, decimal monthlySalary)
{
    MonthlySalary = monthlySalary;
}
```

```
while (true) // simple menu loop with user input and exit condition
{
    Menu();
    string input = Console.ReadLine();
    if (int.TryParse(input, out int choice))
    {
        switch (choice)
        {
            case 1:
                Console.Clear();
                CarMenu();
                Console.WriteLine("Any key to return to main menu: ");
                Console.ReadKey();
                Console.Clear();
                break;
            case 2:
                break;
            default:
                break;
        }
    }
    else { if (input.ToUpper() == "E") { return; } }
}
```

```
TransactionType.DEPOSIT // accessing enumerator
```

```
class Guest : Person // extending class Person
```

```
public Guest(string firstName, string lastName, DateTime checkInDate, DateTime checkOutDate) : base(firstName, lastName)
{
    if (checkInDate >= checkOutDate)
        throw new ArgumentException("Check In date must be before Check Out date");
    CheckIn = checkInDate;
    CheckOut = checkOutDate;
}
```

```
students.Sort((s1, s2) => s1.LastName.CompareTo(s2.LastName)); // sort with labmda
```

```
// Queue's can hold anything we want! First-In-First-Out (FIFO)
Queue<int> queue = new Queue<int>();
queue.Enqueue(1); // add the number 1
queue.Enqueue(2); // add the number 2
queue.Enqueue(3); // add the number 3
Console.WriteLine(queue.Peek()); // Peek() gives us 1 cuz it went in first!
queue.Dequeue(); // removes the first item in the Queue (1) We can also return this value into a variable...
Console.WriteLine(queue.Peek()); // Peek() gives us 2 cuz we took 1 out!
Console.WriteLine(queue.Count); // Counts the things in the Queue!
```

```
// we can also query the database directly to order by something
string query = "SELECT * FROM employees ORDER BY employee_id DESC";
SqlCommand command = new SqlCommand(query, Connection);
// NOTE: we only use SqlDataReader when we are trying to get results back from the database.
using (SqlDataReader reader = command.ExecuteReader())
{
    while (reader.Read())
    {
        int ID = int.Parse(reader["employee_id"].ToString());
        string firstName = reader["first_name"].ToString();
        string lastName = reader["last_name"].ToString();
        int departmentID = int.Parse(reader["department_id"].ToString());
        string gender = reader["gender"].ToString();
        double salary = double.Parse(reader["salary"].ToString());
        string prov = reader["province"].ToString();
        employeeListBox.Items.Add($"{ID,-15}|{firstName,-15}|{lastName,-15}|{departmentID,-15}|{gender,-15}|{salary,-15}|{prov,-15}");
    }
}
```

```
// how to execute a command - DELETE
string query = $"DELETE FROM employees WHERE employee_id = {ID}";
using (SqlCommand command = new SqlCommand(query, Connection))
{
    command.ExecuteNonQuery();
}
```

```
// how to execute a command - INSERT
string query = "INSERT INTO employees (first_name, last_name, department_id, gender, salary, province) VALUES (@fn, @ln, @depID, @gender, @salary, @prov)";
using (SqlCommand command = new SqlCommand(query, Connection))
{
    command.Parameters.AddWithValue("@fn", fn);
    command.Parameters.AddWithValue("@ln", ln);
    command.Parameters.AddWithValue("@depID", depID);
    command.Parameters.AddWithValue("@gender", gender);
    command.Parameters.AddWithValue("@salary", salary);
    command.Parameters.AddWithValue("@prov", prov);
    command.ExecuteNonQuery();
}
```

```
public enum TransactionType
{
    DEPOSIT,
    WITHDRAWAL
}
```

```

namespace ConsoleApp1
{
    internal class Program
    {
        static SqlConnection Connection;
        static string connectionString = @"Data Source=.\\SQLEXPRESS;Initial Catalog=toms_plumbing; Integrated Security=True";
        static void Main(string[] args)
        {
            try
            {
                Connection = new SqlConnection();
                Connection.ConnectionString = connectionString;
                Connection.Open();

                string query = "SELECT * FROM employees";
                SqlCommand cmd = new SqlCommand(query, Connection);

                while (true)
                {
                    using (SqlDataReader reader = cmd.ExecuteReader())
                    {
                        while (reader.Read())
                        {
                            string firstName = reader["first_name"].ToString();
                            string lastName = reader["last_name"].ToString();
                            double salary = double.Parse(reader["salary"].ToString());
                            Console.WriteLine($"{firstName,-20}{lastName,-20}{salary.ToString("F2",-20)}");
                        }
                    }
                    Console.WriteLine("1. Add Employee");
                    Console.WriteLine("Enter Option: ");

                    string input = Console.ReadLine();
                    if (input.ToUpper() == "E")
                    {
                        break;
                    }
                    if (double.TryParse(input, out double d1))
                    {
                        switch (d1)
                        {
                            case 1:
                                Console.Clear();
                                Console.WriteLine("Enter First Name: ");
                                string firstName = Console.ReadLine();
                                Console.WriteLine("Enter Last Name: ");
                                string lastName = Console.ReadLine();
                                Console.WriteLine("Enter Department ID: ");
                                string department = Console.ReadLine();
                                Console.WriteLine("Enter Gender: ");
                                string gender = Console.ReadLine();
                                Console.WriteLine("Enter Salary: ");
                                string salary = Console.ReadLine();
                                Console.WriteLine("Enter Province: ");
                                string prov = Console.ReadLine();
                                InsertEmployee(firstName, lastName, department, gender, salary, prov);
                                break;
                            default:
                                break;
                        }
                    }
                }
            }
            catch (Exception ex) { }
        }

        static void InsertEmployee(string fname, string lname, string dep, string gen, string salary, string pro)
        {
            string query = "INSERT INTO employees(first_name, last_name, department_id, gender, salary, province) VALUES (@fn,@ln,@dep,@gen,@salary,@pro)";

            using (SqlCommand command = new SqlCommand(query, Connection))
            {
                command.Parameters.AddWithValue("@fn", fname);
                command.Parameters.AddWithValue("@ln", lname);
                command.Parameters.AddWithValue("@dep", dep);
                command.Parameters.AddWithValue("@gen", gen);
                command.Parameters.AddWithValue("@salary", salary);
                command.Parameters.AddWithValue("@pro", pro);
                command.ExecuteNonQuery();
            }
        }
    }
}

```

```

// Stack's can hold anything we want! Last-In-Last-Out (LIFO)
Stack<int> stack = new Stack<int>();
stack.Push(1); // add the number 1
Console.WriteLine(stack.Peek()); // Peek() gives us 1 cuz its at the top of the stack
stack.Push(2);
Console.WriteLine(stack.Peek()); // Now we get 2 because its at the top now instead of 1
int pop = stack.Pop(); // This removes the top item and we can return it, store it...
Console.WriteLine(pop); // This will give us 2
Console.WriteLine(stack.Peek()); // Now 1 is at the top
Console.WriteLine("Items Left: " + stack.Count); // Counts the items in the stack... should be 1

```

```

// we can loop through Stack's. Use var for objects i think
int[] numbers = new int[] { 8,2,3,4,7,6,2};
Stack<int> myStack = new Stack<int>();

foreach (int number in numbers)
{
    myStack.Push(number);
}

foreach(int t in myStack)
{
    Console.WriteLine(t);
}

```